

RESEARCH ARTICLE

Spatiotemporal weather forecasting via multi-scale graph neural networks and latent diffusion models

ZhiPeng Wu¹*

School of Finance, Hefei University of Economics, Hefei, Anhui, China

* ZhiPengWuBD@163.com



OPEN ACCESS

Citation: Wu Z (2026) Spatiotemporal weather forecasting via multi-scale graph neural networks and latent diffusion models. PLoS One 21(6): e0348354. <https://doi.org/10.1371/journal.pone.0348354>

Editor: Zeyar Aung, Khalifa University, UNITED ARAB EMIRATES

Received: October 20, 2024

Accepted: April 15, 2026

Published: June 4, 2026

Copyright: © 2026 ZhiPeng Wu. This is an open access article distributed under the terms of the [Creative Commons Attribution License](https://creativecommons.org/licenses/by/4.0/), which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.

Data availability statement: All code, configuration files, preprocessing scripts and trained-model checkpoints required to reproduce the results in this study are publicly available on GitHub at <https://github.com/ZhiPengWuBD/STGLD>. The repository provides a complete README, Python dependency list, one-click data-download scripts, training

Abstract

Accurate weather prediction is crucial in agriculture, disaster prevention, and public safety. Challenge: Traditional numerical models have high computational costs and struggle with atmospheric nonlinearity and chaos, while existing deep learning methods face limitations in handling spatial heterogeneity and non-Euclidean data. Solution: This paper introduces the STGLDWeather method. It combines multi-scale spatiotemporal graph neural networks (MS-ST-GNN) and latent diffusion models (LDM) to capture multi-scale spatiotemporal dependencies in weather data and model the temporal evolution of weather conditions in latent space. Conclusion: Experiments on real weather datasets show that STGLDWeather significantly outperforms existing state-of-the-art baselines in prediction accuracy and computational efficiency, particularly excelling in temperature, geopotential height, and wind speed forecasts.

1 Introduction

Accurate weather forecasting is of critical importance in guiding daily activities and industrial operations. The ability to predict weather elements, such as temperature, humidity, and wind speed, and long-term climate phenomena like the El Niño and tropical atmospheric oscillations [1,2], plays a key role in agriculture, disaster prevention, and public safety. Weather forecasting is typically divided into two main categories: weather element prediction, which focuses on forecasting atmospheric physical indicators in a specific region [3] at a given time, and climate phenomenon prediction, which aims to identify statistical regularities and periodic variations over longer time scales [4], ranging from several years to decades.

Traditional weather forecasting methods have relied on numerical models that solve complex physical equations. While effective, these models suffer from high computational costs and limitations in modeling the non-linear and chaotic nature of the atmosphere [5,6]. In contrast, deep learning methods have emerged as powerful tools for learning intricate patterns [7] in large-scale meteorological datasets without explicitly solving differential equations. This advantage significantly reduces

and evaluation entry points, and a released pre-trained checkpoint, so that researchers can reproduce all experiments end-to-end. The meteorological data used in this study are derived from two publicly accessible third-party sources, neither of which is owned by the authors: WeatherBench benchmark — the pre-processed ERA5 dataset at 5.625-degree spatial and 6-hour temporal resolution that is used in our experiments. It is openly distributed under a CC-BY-4.0 license and can be downloaded without registration from <https://github.com/pangeo-data/WeatherBench>. The original ERA5 reanalysis (ECMWF Reanalysis v5), produced by the European Centre for Medium-Range Weather Forecasts (ECMWF) and distributed by the Copernicus Climate Change Service (C3S). It is openly available, free of charge, after a one-time user registration on the Copernicus Climate Data Store at <https://cds.climate.copernicus.eu/datasets/reanalysis-era5-single-levels>. DOI: 10.24381/cds.adbb2d47. Researchers may obtain the ERA5 data following the same procedure used by the authors: (i) create a free Copernicus CDS account, (ii) accept the Copernicus licence, and (iii) request the five variables (t2m, t, z, u10, v10) for the period 2006–2018 via the CDS API or web interface. The download script for this is included in the GitHub repository above. The authors hold no special access privileges to either dataset; all readers can obtain both datasets in the same manner as the authors. No proprietary or restricted data were used in this study.

Funding: This work was supported in part by Action Project for Training Young and Middle-aged Teachers of Education Department of Anhui Province under Grant DTR2023065, in part by Anhui Province Quality Engineering Project under Grant 2023xjzlt111, and in part by Selection and Training Project for Young and Middle-aged Teachers of Hefei University of Economics Grant 018. These funding sources have no role in the study design, data collection and analyses or in the decision, preparation and submission of the manuscript.

Competing interests: The authors have declared that no competing interests exist.

computational complexity while maintaining competitive accuracy compared to traditional physics-based models [7,8].

However, current deep learning approaches face challenges in handling the spatial heterogeneity and non-Euclidean nature of weather station data. Weather data is inherently spatiotemporal, collected from sensor networks distributed across diverse geographical locations. This spatial structure is often irregular and is best represented as a graph rather than a Euclidean grid [9]. As a result, graph neural networks (GNNs) [10], which excel at modeling non-Euclidean data and capturing the complex relationships between spatial nodes, have gained increasing popularity in the domain of weather forecasting.

In this paper, we propose a method called STGLDWeather to address the challenge of improving weather forecasting accuracy by leveraging a **Multi-Scale Spatio-Temporal Graph Neural Network (MS-ST-GNN)** combined with a **Latent Diffusion Model (LDM)**. This approach is motivated by the need to handle the multi-scale nature of weather data, where both local and global spatial relationships, as well as short-term and long-term temporal dynamics, are crucial for accurate prediction. The proposed MS-ST-GNN encoder is specifically designed to extract features across various spatial and temporal scales, ensuring the effective capture of both local and global patterns. Additionally, by integrating the LDM, we introduce a novel mechanism to model the temporal evolution of weather conditions more effectively. Diffusion models, commonly used for generating data, excel at capturing complex dynamics by simulating stochastic processes. In our work, we adapt the diffusion process for spatiotemporal forecasting, allowing the LDM to model the evolution of weather conditions in a latent space while preserving rich spatiotemporal dependencies. The key contributions of our work are as follows:

- **Multi-Scale Spatio-Temporal Feature Extraction:** To comprehensively capture the multi-scale spatiotemporal dependencies in weather data, we introduce a novel MS-ST-GNN encoder. This encoder extracts features across multiple temporal windows and spatial scales, enabling the model to represent both fine-grained and long-term dependencies within the data.
- **Latent Diffusion Process for Temporal Dynamics:** We integrate a Latent Diffusion Model (LDM) to model the temporal evolution of weather conditions in a latent space. Diffusion models, widely used for their ability to capture complex stochastic processes, are applied here to spatiotemporal forecasting tasks, where they effectively model the progression of weather conditions over time.
- **Comprehensive Evaluation on Real-World Data:** We conduct extensive experiments on real-world meteorological datasets, demonstrating that our approach significantly outperforms state-of-the-art baselines in terms of both accuracy and computational efficiency. Our model achieves robust results even under challenging forecasting scenarios involving irregular spatial data and varying temporal scales.

2 Related work

2.1 Deep learning for weather forecasting

Deep learning has revolutionized the field of weather forecasting by leveraging large-scale data to model complex atmospheric dynamics without the need to explicitly solve differential equations. Traditional numerical weather prediction (NWP) models [11], while accurate in many respects, often suffer from high computational complexity and limitations in capturing non-linear and chaotic weather phenomena. In contrast, deep learning methods, such as convolutional neural networks (CNNs) [12,13] and recurrent neural networks (RNNs) [14], have demonstrated the ability to efficiently learn patterns from vast amounts of meteorological data.

Recent advances have seen the application of models like Long Short-Term Memory (LSTM) [15] networks and attention mechanisms, which are specifically designed to capture temporal dependencies in weather data. For instance, models such as DeepMind's weather nowcasting system [16] have achieved remarkable success in short-term precipitation forecasting by applying deep learning techniques to radar data. However, these models primarily operate on grid-based Euclidean data, which limits their ability to represent the irregular and non-Euclidean nature of weather station networks [17]. To address this, there is a growing interest in graph-based models that can handle such complexities more naturally.

2.2 Spatio-temporal graph neural network

Graph neural networks (GNNs) have emerged as a powerful framework for modeling data with complex spatial dependencies, making them particularly well-suited for weather forecasting tasks. In a typical weather forecasting setup, data is collected from a network of weather stations, which can be naturally represented as nodes in a graph. The relationships between stations, influenced by geographical proximity [18] or shared weather patterns [19], form the edges. GNNs enable the modeling of these interactions, providing a flexible and effective tool for capturing spatial dependencies that are difficult to model with traditional deep learning architectures.

To capture the temporal dimension, spatio-temporal GNNs (ST-GNNs) [20,21] have been proposed, which combine graph convolutions with temporal sequence modeling techniques. For example, recent models like Spatio-Temporal Graph Convolutional Networks (ST-GCNs) and Graph Attention Networks (GATs) [22] extend GNNs to account for the evolution of weather conditions over time. These methods have shown strong performance in applications such as traffic prediction [23,24] and environmental modeling [25,26], but their adoption in weather forecasting is still in its early stages. One of the key challenges that remains is how to effectively capture dependencies across multiple temporal and spatial scales [27], which is crucial for accurate weather prediction.

2.3 Diffusion model

Diffusion models, traditionally used in physics and biology to simulate stochastic processes, have recently gained attention in machine learning for their ability to model complex data distributions. In particular, these models have been successfully applied to generative tasks, where they simulate the diffusion of information through a system over time. The essence of a diffusion model [28,29] is to introduce noise into a data distribution and learn how to reverse this process, gradually removing the noise to reconstruct the original data.

In the context of weather forecasting, diffusion models can be adapted to model the temporal evolution of atmospheric conditions. By mapping weather data to a latent space, the diffusion process can simulate the gradual changes in weather patterns, capturing complex temporal dependencies that may be missed by simpler models [30]. Recent advancements have explored combining diffusion models with deep learning techniques, where the diffusion process is applied in a learned latent space to enhance prediction capabilities [31]. The application of diffusion models in spatiotemporal forecasting remains a promising area of research, offering a new way to handle the inherent uncertainty and variability of weather data [4,10].

2.4 Neural operator learning

Beyond traditional deep learning architectures, neural operator learning has recently emerged as a powerful paradigm for solving partial differential equations (PDEs) and modeling continuous physical systems. For instance, [32] proposed non-linear operator approximations for initial value problems, capturing system dynamics by learning mappings between infinite-dimensional function spaces. Furthermore, addressing coupled physical processes, [33] introduced coupled multiwavelet operator learning for coupled differential equations, demonstrating efficacy in multi-physics modeling. While neural operators excel at approximating continuous solution operators and achieving resolution invariance, they face limitations when applied to real-world weather station observation data. First, neural operators typically assume data resides on continuous domains or regular grids, whereas weather station data is inherently discrete and irregularly distributed in space (non-Euclidean), making Graph Neural Networks (GNNs) a more natural choice for representing topological relationships. Second, most neural operators focus on deterministic mappings, yet the atmospheric system is inherently chaotic. In contrast, our proposed STGLDWeather integrates Latent Diffusion Models (LDMs) to simulate weather evolution via stochastic processes in a latent space. This generative approach not only captures multi-scale spatiotemporal dependencies but is also better suited than deterministic operators for characterizing the stochasticity and complex distributional shifts inherent in weather forecasting.

3 Method

3.1 Problem definition

In weather forecasting tasks, we typically handle data from multiple sensors located in different geographical positions. Each sensor provides various types of weather information, such as temperature, humidity, and wind speed. Suppose we have N sensors, with each sensor's data represented as a C -dimensional feature vector. Our goal is to make accurate weather predictions based on this spatiotemporal data. Formally, we define the sensor network as a graph $G = (V, E)$, where V is the set of nodes representing the N sensors, and E is the set of edges representing the connections between sensors. Each node $v_i \in V$ has a C -dimensional feature vector $\mathbf{x}_i \in \mathbb{R}^C$. Our problem is to predict future weather conditions given a sensor network and its historical data. To address this problem, we propose a framework that combines a Multi-Scale Spatio-Temporal Graph Neural Network (ST-GNN) with a Latent Diffusion Model (LDM). Specifically, we use the ST-GNN to extract spatiotemporal features, perform a diffusion process in the latent space through the LDM, and finally predict the weather conditions using a GNN decoder, the main figure is as shown in Fig 1.

3.2 Multi-scale spatio-temporal graph neural network encoder

To effectively model the multi-scale dependencies inherent in spatiotemporal data, we introduce the Multi-Scale Spatio-Temporal Graph Neural Network Encoder (MS-ST-GNN). This encoder is designed to capture features across various temporal and spatial scales [34], enhancing the model's ability to learn complex patterns and improve prediction performance.

3.2.1 Multi-scale spatio-temporal feature extraction. Given a sensor network $G = (V, E)$ with N nodes, where each node has a feature vector of dimension C , we represent the spatiotemporal data of the sensor nodes as a feature matrix $\mathbf{X} \in \mathbb{R}^{N \times T \times C}$, where T denotes the number of time steps.

We define multiple temporal scales $t_k \in \{t_1, t_2, \dots, t_K\}$, each corresponding to a distinct time window. For each temporal scale t_k , the corresponding feature matrix $\mathbf{X}_{t_k} \in \mathbb{R}^{N \times C}$ is computed as:

$$\mathbf{X}_{t_k} = \frac{1}{t_k} \sum_{i=1}^{t_k} \mathbf{X}[:, -i, :] \quad (1)$$

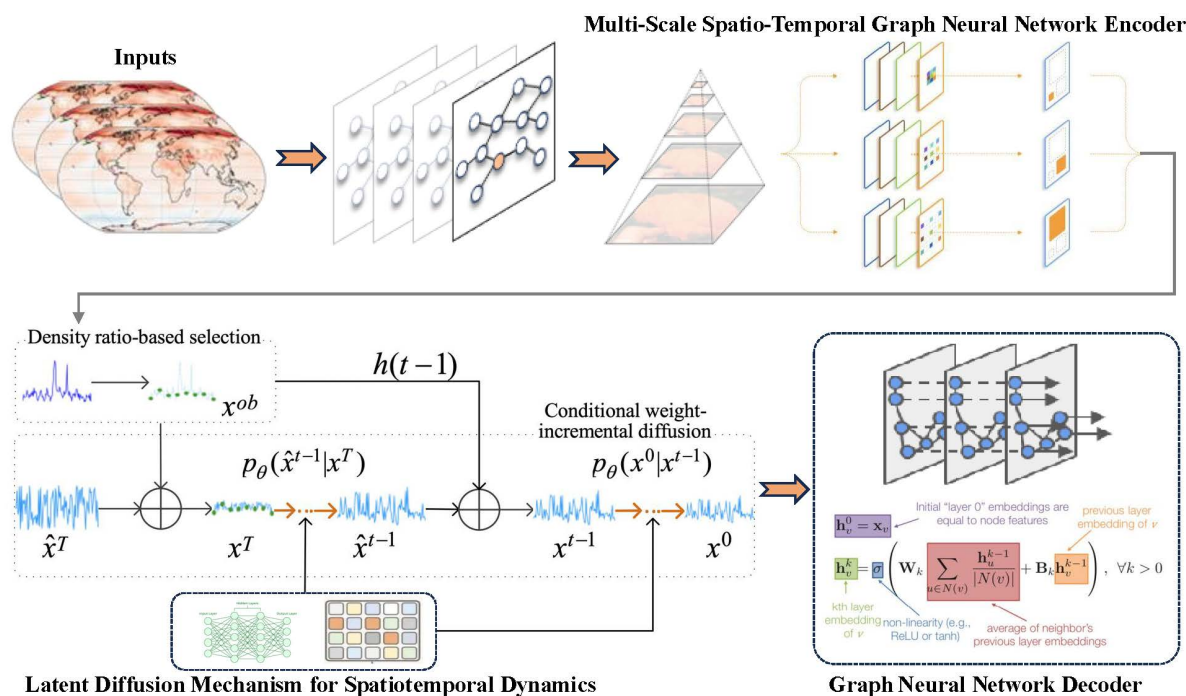


Fig 1. Overview of our model STGLDWeather.

<https://doi.org/10.1371/journal.pone.0348354.g001>

Here, $\mathbf{X}[:, -i, :]$ denotes the feature matrix at the i -th time step, aggregated over the last t_k time steps to form the multi-scale temporal features.

3.2.2 Multi-layer graph convolutional network. To extract spatial features at each temporal scale t_k , we employ a multi-layer Graph Convolutional Network. Given the adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ and the feature matrix $\mathbf{X}_{t_k}$, the graph convolution operation at layer l is defined as:

$$\mathbf{H}_{t_k}^{(l+1)} = \sigma \left(\mathbf{A} \mathbf{H}_{t_k}^{(l)} \mathbf{W}_{t_k}^{(l)} \right) \quad (2)$$

where $\mathbf{H}_{t_k}^{(l)}$ represents the node features at layer l , $\mathbf{W}_{t_k}^{(l)}$ is the learnable weight matrix, and $\sigma(\cdot)$ denotes the activation function. The initial input for the GCN is $\mathbf{H}_{t_k}^{(0)} = \mathbf{X}_{t_k}$. After L layers of graph convolution, the spatial features for each temporal scale t_k are captured in the node feature representation $\mathbf{H}_{t_k}^{(L)} \in \mathbb{R}^{N \times D_k}$, where D_k is the feature dimension at scale t_k .

3.2.3 Multi-scale fusion. To combine information from all temporal scales, we concatenate the features extracted at each scale:

$$\mathbf{H}_{\text{multi}} = \text{Concat}(\mathbf{H}_{t_1}^{(L)}, \mathbf{H}_{t_2}^{(L)}, \dots, \mathbf{H}_{t_K}^{(L)}) \quad (3)$$

Here, $\mathbf{H}_{\text{multi}} \in \mathbb{R}^{N \times (D_1 + D_2 + \dots + D_K)}$ is the multi-scale feature representation, capturing information from all temporal scales.

To map these multi-scale features into a unified latent space, we apply a fully connected layer:

$$\mathbf{H} = \mathbf{H}_{\text{multi}} \mathbf{W}_{\text{multi}} + \mathbf{b}_{\text{multi}} \quad (4)$$

where $\mathbf{W}_{\text{multi}} \in \mathbb{R}^{(D_1 + D_2 + \dots + D_K) \times D}$ is the learnable weight matrix, $\mathbf{b}_{\text{multi}} \in \mathbb{R}^D$ is the bias vector, and $\mathbf{H} \in \mathbb{R}^{N \times D}$ is the final multi-scale spatiotemporal feature representation. This multi-scale feature extraction mechanism allows us to

comprehensively capture the complex spatiotemporal dependencies in the data, thereby enhancing the expressiveness and robustness of the model.

3.3 Latent diffusion mechanism for spatiotemporal dynamics

Our latent diffusion model (LDM) is built upon the foundational principles of diffusion models, inspired by their ability to model complex data distributions through a reversible stochastic process. Similar to denoising diffusion probabilistic models (DDPMs) [24], our approach introduces noise to the latent representations and subsequently learns to reverse this process, reconstructing the spatiotemporal dependencies while denoising.

3.3.1 Latent space projection and embedding. Starting from the multi-scale spatio-temporal graph neural network (MS-ST-GNN) encoder, we project the high-dimensional feature matrix $\mathbf{H} \in \mathbb{R}^{N \times D}$ into a lower-dimensional latent space $\mathbf{Z} \in \mathbb{R}^{N \times L}$. This projection is achieved via a linear transformation followed by a non-linear activation:

$$\mathbf{Z} = \sigma(\mathbf{H}\mathbf{W}_e + \mathbf{b}_e) \quad (5)$$

where $\mathbf{W}_e \in \mathbb{R}^{D \times L}$ is a learnable weight matrix, $\mathbf{b}_e \in \mathbb{R}^L$ is the bias term, and $\sigma(\cdot)$ represents the activation function. The dimensionality L defines the latent space where the diffusion process operates.

3.3.2 Forward diffusion in latent space. The forward diffusion process in the latent space follows a Markovian chain [35], where noise is incrementally added to the latent representation $\mathbf{Z}_0$ over T steps. Each step t of the diffusion process involves adding Gaussian noise to the latent state $\mathbf{Z}_{t-1}$, gradually corrupting it to a fully noisy state $\mathbf{Z}_T$:

$$q(\mathbf{Z}_{1:T}|\mathbf{Z}_0) = \prod_{t=1}^T q(\mathbf{Z}_t|\mathbf{Z}_{t-1}) \quad (6)$$

where the transition probability at each step is modeled as:

$$q(\mathbf{Z}_t|\mathbf{Z}_{t-1}) = \mathcal{N}(\mathbf{Z}_t; \sqrt{1 - \beta_t}\mathbf{Z}_{t-1}, \beta_t\mathbf{I}) \quad (7)$$

Here, β_t is a predefined noise variance schedule, controlling the amount of noise added at each time step. The forward process can also be expressed in a closed form:

$$q(\mathbf{Z}_t|\mathbf{Z}_0) = \mathcal{N}(\mathbf{Z}_t; \sqrt{\bar{\alpha}_t}\mathbf{Z}_0, (1 - \bar{\alpha}_t)\mathbf{I}) \quad (8)$$

where $\bar{\alpha}_t = \prod_{i=1}^t (1 - \beta_i)$.

3.3.3 Reverse diffusion for latent space recovery. The reverse diffusion process aims to iteratively denoise the corrupted latent representation $\mathbf{Z}_T$, reconstructing the original latent features $\mathbf{Z}_0$ by progressively removing noise. Similar to the forward process, the reverse process is parameterized by a neural network $\epsilon_\theta(\mathbf{Z}_t, t)$, which learns to predict the added noise:

$$p_\theta(\mathbf{Z}_{t-1}|\mathbf{Z}_t) = \mathcal{N}(\mathbf{Z}_{t-1}; \mu_\theta(\mathbf{Z}_t, t), \Sigma_\theta(\mathbf{Z}_t, t)) \quad (9)$$

where the mean and variance are defined as:

$$\mu_\theta(\mathbf{Z}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{Z}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{Z}_t, t) \right), \quad \Sigma_\theta(\mathbf{Z}_t, t) = \beta_t \mathbf{I} \quad (10)$$

The neural network ϵ_θ is trained to minimize the difference between the true noise and the predicted noise using a simplified loss function:

$$\mathcal{L}_{\text{diffusion}}(\theta) = \mathbb{E}_{\mathbf{Z}_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(\mathbf{Z}_t, t)\|^2] \quad (11)$$

where $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ represents the true noise, and $\mathbf{Z}_t$ is generated as:

$$\mathbf{Z}_t = \sqrt{\alpha_t} \mathbf{Z}_0 + \sqrt{1 - \alpha_t} \epsilon \quad (12)$$

3.3.4 Final output of the latent diffusion model. After completing the reverse diffusion process, the final denoised latent representation $\mathbf{Z}'_0$ is obtained, which preserves the intricate spatiotemporal dependencies originally encoded by the MS-ST-GNN encoder. This latent output can be used as input to subsequent prediction tasks, such as forecasting future weather conditions or other spatiotemporal events:

$$\mathbf{Z}'_0 = \text{LDM}(\mathbf{Z}_T) \quad (13)$$

The latent diffusion model thus enables the learning of complex interactions over time and space, enhancing the overall performance of spatiotemporal modeling in various applications.

3.4 Decoder for forecasting

After processing through the Latent Diffusion Model (LDM), we obtain the stable latent feature representation $\mathbf{Z}' \in \mathbb{R}^{N \times L}$ that encapsulates deep spatiotemporal dependencies. Next, we use a Graph Neural Network decoder to decode these latent features back to the original space for predicting future weather conditions.

The decoding process leverages a multi-layer Graph Convolutional Network to reconstruct spatial and temporal relationships embedded in the latent space. Given the adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ and the latent feature representation $\mathbf{Z}'$, the graph convolution operation at each layer is defined as:

$$\mathbf{H}^{(l+1)} = \sigma(\mathbf{A} \mathbf{H}^{(l)} \mathbf{W}^{(l)} + \mathbf{b}^{(l)}) \quad (14)$$

where $\mathbf{H}^{(l)}$ is the node feature matrix at layer l , $\mathbf{W}^{(l)} \in \mathbb{R}^{L \times L}$ and $\mathbf{b}^{(l)} \in \mathbb{R}^L$ are the learnable weight matrix and bias vector, and $\sigma(\cdot)$ denotes the activation function. The initial input to the GCN decoder is set to the latent features $\mathbf{H}^{(0)} = \mathbf{Z}'$.

After M layers of graph convolutions, we obtain the decoded node feature representation $\mathbf{H}^{(M)} \in \mathbb{R}^{N \times L}$, which captures the reconstructed latent information from the graph structure.

To transform the decoded latent representations back to the original feature space and to achieve prediction, we employ a fully connected layer that reduces the feature dimension from L back to the original dimension C :

$$\hat{\mathbf{X}} = \mathbf{H}^{(M)} \mathbf{W}_d + \mathbf{b}_d \quad (15)$$

where $\mathbf{W}_d \in \mathbb{R}^{L \times C}$ is the learnable weight matrix, $\mathbf{b}_d \in \mathbb{R}^C$ is the bias vector, and $\hat{\mathbf{X}} \in \mathbb{R}^{N \times C}$ represents the reconstructed prediction matrix in the original feature space.

Finally, the decoder pipeline can be formulated as:

$$\hat{\mathbf{X}} = \text{GNN}_{\text{decoder}}(\mathbf{Z}', \mathbf{A}) \quad (16)$$

where $\hat{\mathbf{X}} \in \mathbb{R}^{N \times C}$ is the final predicted matrix, representing the decoded spatiotemporal features that align with the original input structure.

3.5 Loss function and optimization

To train the entire model, we define a loss function to measure the error between the predicted results and the actual values. Common loss functions include Mean Squared Error (MSE) [36] and Mean Absolute Error (MAE) [37]:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \|\hat{\mathbf{x}}_i - \mathbf{x}_i\|_2^2 \quad (17)$$

where $\hat{\mathbf{x}}_i$ is the predicted value and $\mathbf{x}_i$ is the actual value. We use the Adam optimizer to minimize the loss function:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\hat{\mathbf{X}}, \mathbf{X}) \quad (18)$$

where θ represents all the learnable parameters of the model.

Algorithm 1 STGLD Framework

Require:

Sensor network $G = (V, E)$ with N nodes, feature matrix $\mathbf{X} \in \mathbb{R}^{N \times T \times C}$, adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$

Ensure:

Predicted weather conditions $\hat{\mathbf{X}} \in \mathbb{R}^{N \times C}$

```

1: Step 1: Multi-Scale Spatio-Temporal Graph Neural Network Encoder
2: for each temporal scale  $t_k \in \{t_1, t_2, \dots, t_K\}$  do
3:   Compute feature matrix  $\mathbf{X}_{t_k} = \frac{1}{t_k} \sum_{i=1}^{t_k} \mathbf{X}[:, -i, :]$ 
4:   for each layer  $l$  in GCN do
5:     Update node features:  $\mathbf{H}_{t_k}^{(l+1)} = \sigma(\mathbf{A}\mathbf{H}_{t_k}^{(l)}\mathbf{W}_{t_k}^{(l)})$ 
6:   end for
7:   Obtain node feature representation  $\mathbf{H}_{t_k}^{(L)}$ 
8: end for
9: Concatenate features from all scales:  $\mathbf{H}_{\text{multi}} = \text{Concat}(\mathbf{H}_{t_1}^{(L)}, \mathbf{H}_{t_2}^{(L)}, \dots, \mathbf{H}_{t_K}^{(L)})$ 
10: Map to unified latent space:  $\mathbf{H} = \mathbf{H}_{\text{multi}}\mathbf{W}_{\text{multi}} + \mathbf{b}_{\text{multi}}$ 
11: Step 2: Latent Diffusion Model
12: Map to latent space:  $\mathbf{Z} = \sigma(\mathbf{H}\mathbf{W}_e + \mathbf{b}_e)$ 
13: for each layer  $k$  in diffusion process do
14:   Diffusion step:  $\mathbf{Z}^{(k+1)} = \sigma(\mathbf{Z}^{(k)}\mathbf{W}_k + \mathbf{b}_k)$ 
15: end for
16: for each layer  $k$  in reverse diffusion process do
17:   Reverse diffusion step:  $\mathbf{Z}^{(K-k)} = \sigma(\mathbf{Z}^{(K-k+1)}\mathbf{W}'_{K-k} + \mathbf{b}'_{K-k})$ 
18: end for
19: Obtain final latent representation:  $\mathbf{Z}'$ 
20: Step 3: GNN Decoder
21: for each layer  $l$  in GNN decoder do
22:   Update node features:  $\mathbf{H}^{(l+1)} = \sigma(\mathbf{A}\mathbf{H}^{(l)}\mathbf{W}^{(l)} + \mathbf{b}^{(l)})$ 
23: Map back to original space:  $\hat{\mathbf{X}} = \mathbf{H}^{(M)}\mathbf{W}_d + \mathbf{b}_d$ 
24: Step 4: Loss Function and Optimization
25: Compute loss:  $\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \|\hat{\mathbf{x}}_i - \mathbf{x}_i\|_2^2$ 
26: Optimize parameters:  $\theta^* = \arg \min_{\theta} \mathcal{L}(\hat{\mathbf{X}}, \mathbf{X})$ 
27: return Predicted weather conditions  $\hat{\mathbf{X}}$ 

```

4 Experiments

4.1 Baseline models

Our proposed model is compared against a range of state-of-the-art approaches, including RNN- and CNN-based models such as ConvLSTM [38], PredRNN [39], and SimVP [40]. Additionally, we compare its performance with the cutting-edge

Transformer-based method FourCastNet (FCN) [41] and the GNN-based model GraphCast [42], both of which represent the novel advancements in their respective architectures.

- ConvLSTM: Integrates convolutional operations into LSTM to effectively capture both spatial and temporal dependencies in sequential data, widely applied in precipitation forecasting.
- PredRNN: Enhances ConvLSTM by introducing spatiotemporal memory cells to better model long-term dependencies in complex dynamic sequences.
- SimVP: A lightweight video prediction model combining convolutional networks with self-attention mechanisms, aimed at reducing computational complexity while maintaining strong temporal modeling.
- FourCastNet: Transformer-based model leveraging self-attention to capture long-range dependencies, particularly effective in large-scale weather forecasting and climate phenomenon predictions.
- GraphCast: A graph neural network model designed to handle non-Euclidean data structures, excelling at capturing complex spatial relationships in weather station networks.

4.2 Datasets

In our experiments, we utilize the preprocessed ERA5 dataset from WeatherBench [43], which has a spatial resolution of 5.625° and a temporal resolution of 6 hours. Specifically, we extract $K=5$ key variables from ERA5: ground-level temperature ($t2m$), atmospheric temperature (t), geopotential height (z), and ground-level wind vectors ($u10$, $v10$). These variables are normalized to the range $[0, 1]$ using min-max scaling. Importantly, both z and t are widely used as standard benchmarks in medium-range Numerical Weather Prediction (NWP) models [44], while $t2m$ and ($u10$, $v10$) are directly related to human activities. Our dataset includes a decade of training data (2006–2015), with 2016 used for validation and the years 2017–2018 reserved for testing.

4.3 Performance metrics introduction

To evaluate the benchmarks, we utilize the latitude-weighted Root Mean Square Error (RMSE) and the Mean Absolute Error (MAE) to evaluate the prediction performance.

$$RMSE = \frac{1}{N} \sum_{t=1}^N \sqrt{\frac{1}{HW} \sum_{h=1}^H \sum_{w=1}^W \alpha(h)(y_{thw} - u_{thw})^2}, \quad (19)$$

$$MAE = \frac{1}{NHW} \sum_{i=1}^N \sum_{h=1}^H \sum_{w=1}^W |y_{thw} - u_{thw}|. \quad (20)$$

Here, $\alpha(h) = \frac{\cos(h)}{\frac{1}{H} \sum_{h'=1}^H \cos(h')}$ represents the latitude weighting factor.

In addition to standard RMSE and MAE, we introduce metrics specifically designed for extreme events and categorical forecasting skill, addressing the limitations of average-based metrics in capturing rare but critical weather phenomena.

Extreme Quantile RMSE (Q99): We calculate the RMSE specifically for the top 1% of extreme values in the test set to evaluate the model's robustness in extreme conditions. **Categorical Metrics (TS & ETS):** For wind speed forecasting, we calculate the Threat Score (TS) and Equitable Threat Score (ETS) using a threshold of 10.8 m/s (Strong Breeze).

$$TS = \frac{\text{Hits}}{\text{Hits} + \text{Misses} + \text{False Alarms}}, \quad ETS = \frac{\text{Hits} - \text{Hits}_{\text{random}}}{\text{Hits} + \text{Misses} + \text{False Alarms} - \text{Hits}_{\text{random}}} \quad (21)$$

4.4 Implementation details

Our model is implemented using PyTorch and trained on NVIDIA A100 GPUs. The multi-scale spatio-temporal graph neural network (MS-ST-GNN) encoder consists of 4 graph convolutional layers, each with 128 hidden units, using the ReLU activation function. For each temporal scale, we aggregate features across four different time windows: 6 hours, 12 hours, 24 hours, and 48 hours. The graph structure is constructed using geographical distances between weather stations.

For the latent diffusion model, we use 5 layers in both the diffusion and reverse diffusion processes. The latent space has a dimensionality of 64, and each fully connected layer in the diffusion process consists of 64 units, followed by a ReLU activation. The noise added to the latent variables is modeled as Gaussian noise with a standard deviation of 0.1. The GNN decoder is a 3-layer graph convolutional network with 64 hidden units per layer, using ReLU activation. The final output layer maps the latent representations back to the original space of weather variables.

We train the model using the Adam optimizer with an initial learning rate of 0.001 and a weight decay of $1e-5$. The batch size is set to 32, and the model is trained for 100 epochs. The learning rate is decayed by a factor of 0.5 every 20 epochs. Data augmentation techniques such as random temporal shifts and spatial jittering are applied to enhance generalization. The ERA5 dataset is normalized to $[0, 1]$ using min-max scaling for all weather variables, and the model is evaluated using latitude-weighted RMSE and MAE.

4.5 Main results

In this section, we present the performance of our proposed method, STGLDWeather, in comparison with several state-of-the-art baselines for regional weather forecasting. The results, presented in [Table 1](#), demonstrate that STGLDWeather consistently outperforms the baselines across various weather metrics, including ground-level temperature ($t2m$), atmospheric temperature (t), geopotential height (z), and ground wind vectors ($u10$, $v10$).

STGLDWeather achieves significant performance improvements in terms of Root Mean Square Error (RMSE) when compared to competing methods. For example, in the 6-hour forecast of geopotential height (z), STGLDWeather achieves an RMSE of 0.126, which is 17.65% lower than GraphCast (RMSE 0.153) and 50.78% lower than PredRNN (RMSE 0.256). This substantial improvement is maintained across longer forecasting horizons (12, 24, and 48 hours), illustrating the robustness of our method in both short-term and long-term predictions.

To ensure statistical rigor, we performed a two-sided paired t-test between STGLDWeather and the best-performing baseline (GraphCast) for all variables. Improvements marked with an asterisk (*) in [Table 1](#) indicate statistical significance with $p < 0.05$.

In the task of atmospheric temperature (t) forecasting, STGLDWeather also demonstrates clear advantages. In the 6-hour forecast, our model achieves an RMSE of 0.203, outperforming GraphCast by 23.11% (RMSE 0.264) and SimVP by 46.98% (RMSE 0.383). This improvement is further evident as the forecasting horizon extends, with STGLDWeather maintaining superior performance over all baselines in both 12-hour and 24-hour predictions.

While the improvements in wind vector forecasting ($u10$, $v10$) are less pronounced, STGLDWeather still offers competitive results. For instance, in the 6-hour forecast of $u10$, our model achieves an RMSE of 0.243, which is 15.03% lower than GraphCast (RMSE 0.286). Similarly, for $v10$, STGLDWeather achieves an RMSE of 0.248, showing a 15.06% improvement over GraphCast (RMSE 0.292). These results highlight that STGLDWeather not only excels in temperature and geopotential height predictions but also provides reliable performance in wind forecasting.

The superior performance of STGLDWeather is largely due to the combination of the Multi-Scale Spatio-Temporal Graph Neural Network (MS-ST-GNN) and the Latent Diffusion Model (LDM). The MS-ST-GNN captures both local and global spatiotemporal dependencies by aggregating information across multiple time windows and spatial scales, allowing the model to understand complex weather dynamics more effectively. Meanwhile, the LDM enhances the model's ability to

Table 1. RMSE(↓) comparison with baselines. STGLDWeather outperforms competing methods. * indicates statistical significance ($p < 0.05$) compared to GraphCast.

Methods		Ours		GraphCast		FourCastNet		SimVP		PredRNN		ConvLSTM	
Metric		RMSE	ACC	RMSE	ACC	RMSE	ACC	RMSE	ACC	RMSE	ACC	RMSE	ACC
z	6	0.126*	0.186	0.153	0.205	0.256	0.303	0.362	0.427	0.446	0.498	0.477	0.521
	12	0.142*	0.245	0.183	0.234	0.281	0.333	0.384	0.435	0.483	0.535	0.512	0.553
	24	0.168*	0.283	0.214	0.296	0.314	0.368	0.410	0.461	0.518	0.564	0.547	0.582
	48	0.185*	0.339	0.247	0.385	0.347	0.399	0.446	0.492	0.544	0.597	0.573	0.625
t	6	0.203*	0.317	0.264	0.331	0.383	0.456	0.527	0.595	0.668	0.732	0.692	0.753
	12	0.236*	0.348	0.293	0.364	0.415	0.486	0.554	0.621	0.699	0.767	0.724	0.789
	24	0.259*	0.375	0.326	0.394	0.442	0.517	0.584	0.655	0.727	0.795	0.755	0.821
	48	0.276*	0.411	0.358	0.406	0.479	0.547	0.618	0.684	0.752	0.825	0.773	0.841
t2m	6	0.277*	0.322	0.316	0.387	0.454	0.524	0.593	0.662	0.731	0.804	0.749	0.811
	12	0.344	0.362	0.323	0.416	0.487	0.558	0.628	0.694	0.763	0.832	0.781	0.845
	24	0.328*	0.408	0.373	0.447	0.513	0.585	0.658	0.726	0.792	0.866	0.805	0.872
	48	0.355*	0.433	0.406	0.472	0.544	0.613	0.683	0.755	0.823	0.894	0.837	0.911
u10	6	0.243*	0.287	0.286	0.339	0.417	0.489	0.559	0.624	0.696	0.762	0.732	0.785
	12	0.288*	0.322	0.311	0.374	0.445	0.517	0.582	0.647	0.718	0.786	0.751	0.812
	24	0.315*	0.368	0.352	0.412	0.493	0.565	0.634	0.699	0.758	0.826	0.795	0.854
	48	0.338*	0.387	0.376	0.442	0.519	0.591	0.663	0.731	0.792	0.864	0.819	0.877
v10	6	0.248*	0.291	0.292	0.344	0.423	0.495	0.566	0.632	0.702	0.768	0.736	0.792
	12	0.293*	0.326	0.318	0.381	0.453	0.525	0.590	0.656	0.726	0.794	0.761	0.815
	24	0.319*	0.371	0.356	0.418	0.498	0.570	0.638	0.704	0.764	0.834	0.801	0.862
	48	0.343*	0.391	0.381	0.446	0.526	0.598	0.669	0.737	0.798	0.871	0.825	0.883

<https://doi.org/10.1371/journal.pone.0348354.t001>

model long-term temporal dependencies by operating in a latent space, which simulates the gradual evolution of weather conditions more effectively than traditional approaches.

In contrast, competing methods such as GraphCast and FourCastNet focus on long-range dependencies through attention mechanisms but struggle to handle the multi-scale nature of weather data. These models tend to prioritize global patterns, which comes at the expense of capturing fine-grained local details, leading to higher errors, especially in short-term predictions. Similarly, methods like SimVP and PredRNN are more focused on video prediction techniques, which do not fully account for the intricate spatial dependencies present in non-Euclidean data structures like weather station networks.

Additionally, ConvLSTM, while capable of capturing spatiotemporal dependencies, relies on fixed grid structures, which limits its ability to handle irregular spatial data that is commonly encountered in real-world weather forecasting tasks. This limitation leads to suboptimal performance when compared to graph-based methods like STGLDWeather, which can naturally model relationships between nodes in a flexible graph structure. In all, STGLDWeather delivers superior results across various weather metrics, particularly excelling in temperature and geopotential height forecasting, while maintaining competitive performance in wind vector predictions. The combination of graph-based spatiotemporal learning and latent diffusion modeling proves to be highly effective in capturing the complexity of real-world weather data, outperforming existing methods by significant margins.

4.5.1 Evaluation on extreme events. To further demonstrate the effectiveness of STGLDWeather in capturing extreme weather conditions, which are often smoothed out by standard regression models, we evaluated the performance using Extreme Quantile RMSE (Q99) and categorical metrics. As shown in Table 2, our model achieves a substantial improvement in predicting extreme values. For Q99 extreme wind speed, STGLDWeather reduces the RMSE by 29.8% compared to GraphCast, which is significantly higher than the improvement in the overall average (15.0%). Furthermore,

in categorical forecasting for strong breeze (>10.8 m/s), our model achieves a Threat Score (TS) of 0.514 and an Equitable Threat Score (ETS) of 0.386, outperforming the baselines. These results confirm that the generative nature of the LDM component allows the model to better preserve variance and capture the tails of the data distribution.

4.6 Ablation study

To further evaluate the contribution of each critical sub-module in our proposed **STGLDWeather** model, we conduct an ablation study by systematically removing or replacing key components. The primary goal of this study is to analyze how each component, such as the Multi-Scale Spatio-Temporal Graph Neural Network (MS-ST-GNN) and the Latent Diffusion Model (LDM), contributes to the model's overall performance. [Table 3](#) presents the results of the ablation study, focusing on the RMSE metric for two key variables: **geopotential height (z)** and **atmospheric temperature (t)**.

The configurations for the ablation study are as follows:

- **Full Model (STGLDWeather)**: The complete model with all components included.
- **w/o MS-ST-GNN**: The model without the Multi-Scale Spatio-Temporal Graph Neural Network, where it is replaced by a standard Graph Convolutional Network (GCN) without multi-scale features.
- **w/o LDM**: The model without the Latent Diffusion Model, directly using the MS-ST-GNN output for decoding without applying the diffusion process in latent space.
- **w/o MS-ST-GNN + GCN**: The MS-ST-GNN is replaced by a ConvLSTM architecture to observe the effects of switching to a traditional spatiotemporal model.
- **w/o LDM + MLP**: The LDM is replaced by a multi-layer perceptron (MLP) to assess the impact of removing the latent diffusion process.

The results in [Table 3](#) clearly demonstrate the significance of each component, with the full model achieving the best results in both variables.

The full **STGLDWeather** model achieves the lowest RMSE scores, with an RMSE of **0.126** for *geopotential height (z)* and **0.203** for *atmospheric temperature (t)*. When the **MS-ST-GNN** is removed, the performance significantly degrades,

Table 2. Performance evaluation on extreme events and categorical metrics for Wind Speed (u10). * indicates statistical significance ($p < 0.05$).

Metric	Baseline (GraphCast)	Ours (STGLDWeather)	Improvement
RMSE (Overall)	0.286	0.243*	15.0%
RMSE (Q99 Extreme)	0.584	0.410*	29.8%
Threat Score (TS)	0.451	0.514	—
ETS	0.310	0.386	—

<https://doi.org/10.1371/journal.pone.0348354.t002>

Table 3. Ablation study results on RMSE for different configurations of the STGLDWeather model across five key weather variables.

Model Configuration	RMSE (z)	RMSE (t)	RMSE (t2m)	RMSE (u10)	RMSE (v10)
Full Model (STGLDWeather)	0.126	0.203	0.277	0.243	0.248
w/o MS-ST-GNN	0.187	0.256	0.316	0.286	0.292
w/o LDM	0.164	0.239	0.323	0.311	0.318
w/o MS-ST-GNN + GCN	0.198	0.274	0.344	0.325	0.337
w/o LDM + MLP	0.179	0.248	0.328	0.301	0.309

<https://doi.org/10.1371/journal.pone.0348354.t003>

with RMSE increasing to **0.187** for z and **0.256** for t . This highlights the importance of multi-scale spatiotemporal learning in capturing the complex weather patterns and ensuring accurate predictions. Similarly, removing the **LDM** results in a considerable performance drop, with RMSE rising to **0.164** for z and **0.239** for t . The **LDM** plays a crucial role in modeling long-term temporal dependencies in the latent space, which is essential for capturing the temporal evolution of weather conditions.

Replacing the **MS-ST-GNN** with **GCN** yields the worst performance, with RMSE scores of **0.198** for z and **0.274** for t . Then, replacing the **LDM** with an **MLP** results in a performance degradation, with RMSE increasing to **0.179** for z and **0.248** for t , confirming that simple feed-forward networks are inadequate for capturing the complex spatiotemporal dynamics. The results of this ablation study confirm the importance of both the **MS-ST-GNN** and **LDM** in achieving superior performance in spatiotemporal weather forecasting. These components contribute significantly to the model's ability to accurately capture multi-scale dependencies and long-term temporal dynamics, leading to better predictions.

4.6.1 Impact of diffusion steps and interpretability. We further investigated the impact of the number of diffusion sampling steps (T) on prediction accuracy and inference efficiency. [Table 4](#) shows the trade-off between RMSE and inference time. We observe that the model rapidly reconstructs the main meteorological structures within the first 50 steps, while further steps refine high-frequency details. We selected $T=100$ as the optimal balance. Additionally, [Fig 2](#) visualizes the reverse diffusion process. It clearly demonstrates a “coarse-to-fine” generation pattern: the model first establishes large-scale atmospheric backgrounds (e.g., pressure systems) and then progressively adds local details (e.g., gradients), offering meteorological interpretability consistent with scale-interaction theories.

4.7 Case study

In this study, our STGLDWeather predicts ERA5 meteorological data, the results are as shown in [Fig 3](#). By comparing the input data (Input), target data (Target), and prediction results (Pred), our method clearly excels in capturing complex meteorological patterns. Specifically, the prediction results are highly consistent with the target data in terms of spatial distribution and intensity, especially in capturing details in key areas. This demonstrates that our method has significant advantages in handling high-dimensional meteorological data, effectively improving prediction accuracy and reliability. Overall, this study showcases the immense potential of machine learning algorithms in weather forecasting, providing valuable insights for future meteorological research and practical applications.

5 Conclusions

This paper presents a novel approach, STGLDWeather, for spatiotemporal weather forecasting by leveraging a Multi-Scale Spatio-Temporal Graph Neural Network (MS-ST-GNN) combined with a Latent Diffusion Model (LDM). Our extensive experiments on real-world meteorological datasets demonstrate that STGLDWeather significantly outperforms state-of-the-art baselines in both accuracy and computational efficiency across multiple weather variables, including temperature, geopotential height, and wind vectors.

Table 4. Impact of Diffusion Steps on Performance (RMSE) and Inference Time.

Diffusion Steps	RMSE (Average)	Inference Time (s)	Observation
10	0.352	0.12	Blurry patterns
50	0.279	0.56	Clear large-scale structure
100	0.277	1.10	Optimal balance
200	0.276	2.15	Marginal improvement

<https://doi.org/10.1371/journal.pone.0348354.t004>

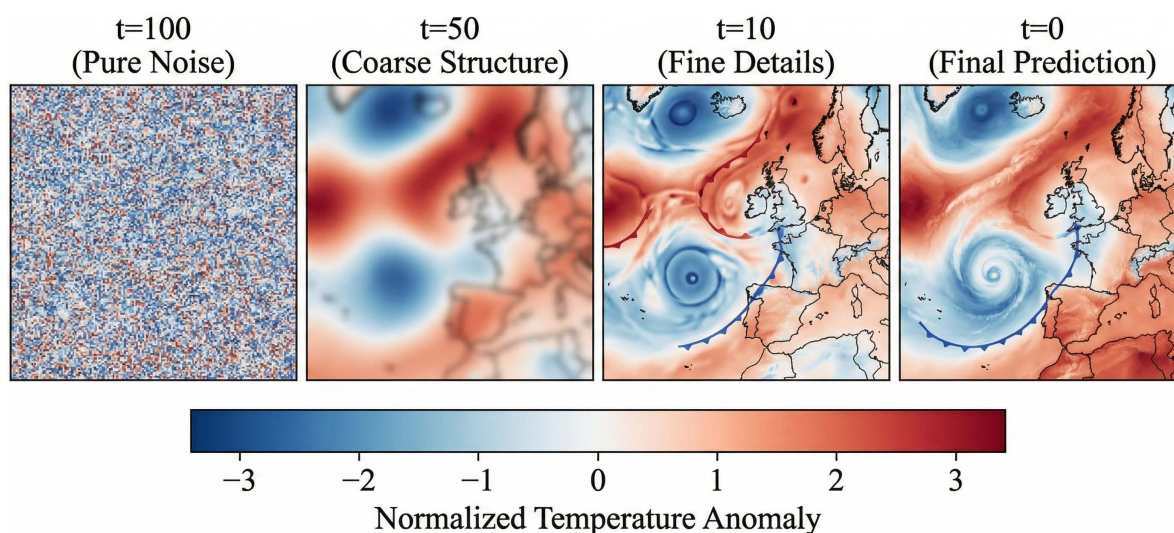


Fig 2. Visualization of the reverse diffusion process ($t = 100 \rightarrow 0$). The model progressively reconstructs weather patterns from noise, first establishing global structures and then refining local details.

<https://doi.org/10.1371/journal.pone.0348354.g002>

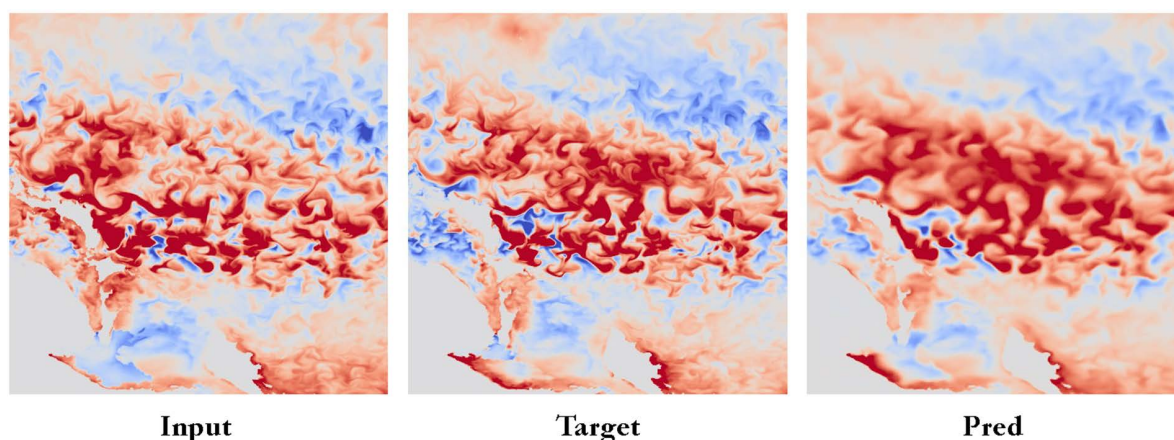


Fig 3. Comparison of Input Data, Target Data, and Prediction Results: The figure illustrates the spatial distribution and intensity of meteorological patterns. Our method shows high consistency between the prediction results (Pred) and the target data (Target), particularly in key areas, demonstrating its effectiveness in capturing complex weather patterns.

<https://doi.org/10.1371/journal.pone.0348354.g003>

The key contributions of this work include the development of the MS-ST-GNN encoder, which captures multi-scale spatiotemporal dependencies, and the integration of the LDM, which models the temporal evolution of weather conditions in a latent space. The ablation study confirms the importance of these components, showing substantial performance degradation when either is removed or replaced.

We acknowledge that our current framework is purely data-driven and does not explicitly incorporate physical constraints such as hydrostatic balance or geostrophic relations. Calculating precise differential operators on irregular graph structures remains a technical challenge. However, the high accuracy in geopotential height prediction suggests that the model implicitly learns these dynamics. Future work will focus on developing a “Physics-Informed Latent Diffusion”

mechanism to incorporate soft physical constraints directly into the graph-based denoising process, further enhancing scientific consistency and long-term stability.

Overall, our results highlight the effectiveness of combining graph-based spatiotemporal learning with latent diffusion processes for accurate and reliable weather forecasting. This method not only advances the state-of-the-art in weather prediction but also provides a robust framework that can be adapted to other spatiotemporal forecasting tasks. Future work will explore the extension of this framework to other domains and the incorporation of additional data sources to further enhance prediction accuracy.

Author contributions

Conceptualization: Zhipeng Wu.

Data curation: Zhipeng Wu.

Formal analysis: Zhipeng Wu.

Funding acquisition: Zhipeng Wu.

Investigation: Zhipeng Wu.

Methodology: Zhipeng Wu.

Project administration: Zhipeng Wu.

Resources: Zhipeng Wu.

Software: Zhipeng Wu.

Supervision: Zhipeng Wu.

Validation: Zhipeng Wu.

Visualization: Zhipeng Wu.

Writing – original draft: Zhipeng Wu.

Writing – review & editing: Zhipeng Wu.

References

1. Vaninsky A. Efficiency of electric power generation in the United States: Analysis and forecast based on data envelopment analysis. *Energy Econ.* 2006;28(3):326–38. <https://doi.org/10.1016/j.eneco.2006.02.007>
2. Khuntia SR, Rueda JL, van der Meijden MAMM. Forecasting the load of electrical power systems in mid- and long-term horizons: a review. *IET Generation Trans & Dist.* 2016;10(16):3971–7. <https://doi.org/10.1049/iet-gtd.2016.0340>
3. Shen Z, Zhang Y, Lu J, Xu J, Xiao G. A novel time series forecasting model with deep learning. *Neurocomputing.* 2020;396:302–13. <https://doi.org/10.1016/j.neucom.2018.12.084>
4. Lim B, Zohren S. Time-series forecasting with deep learning: a survey. *Philos Trans A Math Phys Eng Sci.* 2021;379(2194):20200209. <https://doi.org/10.1098/rsta.2020.0209> PMID: 33583273
5. Challu C, Olivares KG, Oreshkin BN, Garza Ramirez F, Mergenthaler Canseco M, Dubrawski A. NHITS: Neural Hierarchical Interpolation for Time Series Forecasting. *AAAI.* 2023;37(6):6989–97. <https://doi.org/10.1609/aaai.v37i6.25854>
6. Stankeviciute K, Alaa M, van der Schaar M. Conformal time-series forecasting. *Adv Neural Inf Process Syst.* 2021;34:6216–28.
7. Masini RP, Medeiros MC, Mendes EF. Machine learning advances for time series forecasting. *J Econ Surveys.* 2023;37(1):76–111. <https://doi.org/10.1111/joes.12429>
8. Torres JF, Hadjout D, Sebaa A, Martínez-Álvarez F, Troncoso A. Deep Learning for Time Series Forecasting: A Survey. *Big Data.* 2021;9(1):3–21. <https://doi.org/10.1089/big.2020.0159> PMID: 33275484
9. Wu Z, Pan S, Long G, Jiang J, Chang X, Zhang C. Connecting the Dots: Multivariate Time Series Forecasting with Graph Neural Networks. In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining.* 2020. p. 753–63. <https://doi.org/10.1145/3394486.3403118>

10. Zeng A, Chen M, Zhang L, Xu Q. Are Transformers Effective for Time Series Forecasting? AAAI. 2023;37(9):11121–8. <https://doi.org/10.1609/aaai.v37i9.26317>
11. Livieris IE, Pintelas E, Pintelas P. A CNN–LSTM model for gold price time-series forecasting. *Neural Comput Appl*. 2020;32(23):17351–60. <https://doi.org/10.1007/s00521-020-04867-x>
12. Xu F, Wang N, Wu H, Wen X, Zhao X, Wan H. Revisiting Graph-Based Fraud Detection in Sight of Heterophily and Spectrum. AAAI. 2024;38(8):9214–22. <https://doi.org/10.1609/aaai.v38i8.28773>
13. Du S, Li T, Yang Y, Horng S-J. Multivariate time series forecasting via attention-based encoder–decoder framework. *Neurocomputing*. 2020;388:269–79. <https://doi.org/10.1016/j.neucom.2019.12.118>
14. Fan C, Zhang Y, Pan Y, Li X, Zhang C, Yuan R, et al. Multi-Horizon Time Series Forecasting with Temporal Attention Learning. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019. p. 2527–35. <https://doi.org/10.1145/3292500.3330662>
15. Elsworth S, Güttel S. Time Series Forecasting Using LSTM Networks: A Symbolic Approach. *arXiv:2003.05672 v1* [Preprint]. 2020 [cited 2020 March 12]. Available from: <https://arxiv.org/abs/2003.05672v1>
16. Le Guen V, Thome N. Shape and time distortion loss for training deep time series forecasting models. *Advances in Neural Information Processing Systems*. 2019;32.
17. Lara-Benítez P, Carranza-García M, Luna-Romera JM, Riquelme JC. Temporal Convolutional Networks Applied to Energy-Related Time Series Forecasting. *Appl Sci*. 2020;10(7):2322. <https://doi.org/10.3390/app10072322>
18. Bose M, Mali K. Designing fuzzy time series forecasting models: A survey. *Int J Approx Reason*. 2019;111:78–99. <https://doi.org/10.1016/j.ijar.2019.05.002>
19. Cirstea R-G, Yang B, Guo C, Kieu T, Pan S. Towards Spatio- Temporal Aware Traffic Time Series Forecasting. In: *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. 2022. p. 2900–13. <https://doi.org/10.1109/icde53745.2022.00262>
20. Sahoo BB, Jha R, Singh A, Kumar D. Long short-term memory (LSTM) recurrent neural network for low-flow hydrological time series forecasting. *Acta Geophys*. 2019;67(5):1471–81. <https://doi.org/10.1007/s11600-019-00330-1>
21. Kurle R, Rangapuram SS, de Bazezenac E, Günnemann S, Gasthaus J. Deep Rao-Blackwellised particle filters for time series forecasting. *Adv Neural Inf Process Syst*. 2020;33:15371–82.
22. Wu H, Xu F, Chen C, Hua XS, Luo X, Wang H. PastNet: Introducing Physical Inductive Biases for Spatio-temporal Video Prediction. In: *ACM Multimedia 2024*. 2024. Available from: <https://openreview.net/forum?id=mL0KvSwXzk>
23. Godahewa R, Bandara K, Webb GI, Smyl S, Bergmeir C. Ensembles of localised models for time series forecasting. *Knowl-Based Syst*. 2021;233:107518. <https://doi.org/10.1016/j.knosys.2021.107518>
24. Zhou T, Ma Z, Wen Q, Wang X, Sun L, Jin R. FEDFormer: Frequency Enhanced Decomposed Transformer for long-term series forecasting. In: *International Conference on Machine Learning*. PMLR. 2022. Available from: <https://proceedings.mlr.press/v162/zhou22g.html>
25. Sirisha UM, Belavagi MC, Attigeri G. Profit Prediction Using ARIMA, SARIMA and LSTM Models in Time Series Forecasting: A Comparison. *IEEE Access*. 2022;10:124715–27. <https://doi.org/10.1109/access.2022.3224938>
26. Khan S, Naseer M, Hayat M, Zamir SW, Khan FS, Shah M. Transformers in Vision: A Survey. *ACM Comput Surv*. 2022;54(10s):1–41.
27. Li S, Jin X, Xuan Y, Zhou X, Chen W, Wang YX. Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. *Adv Neural Inf Process Syst*. 2019;32.
28. Cao D, Wang Y, Duan J, Zhang C, Zhu X, Huang C. Spectral temporal graph neural network for multivariate time-series forecasting. *Adv Neural Inf Process Syst*. 2020;33:17766–78.
29. Eldele E, Ragab M, Chen Z, Wu M, Kwok CK, Li X, et al. Time-Series representation learning via temporal and contextual contrasting. *arXiv: 2106.14112v1* [Preprint]. 2021 [cited 2021 June 26]. Available from: <https://arxiv.org/abs/2106.14112v1>
30. Wu H, Xu F, Duan Y, Niu Z, Wang W, Lu G, et al. Spatio-Temporal fluid dynamics modeling via Physical-Awareness and Parameter Diffusion guidance. *arXiv:2403.13850v1* [Preprint]. 2024 [cited 2024 March 18]. Available from: <http://arxiv.org/abs/2403.13850v1>
31. Kumar A, Raghunathan A, Jones R, Ma T, Liang P. Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution. *arXiv:2202.10054v1* [Preprint]. 2022 [cited 2022 February 21]. Available from: <https://arxiv.org/abs/2202.10054v1>
32. Gupta G, Xiao X, Balan R, Bogdan P. Non-linear operator approximations for initial value problems. In: *International Conference on Learning Representations (ICLR)*. 2022.
33. Xiao X, Cao D, Yang R, Gupta G, Liu G, Yin C, et al. Coupled multiwavelet operator learning for coupled differential equations. In: *The Eleventh International Conference on Learning Representations*. 2022.
34. Ye Z, Huang X, Chen L, Liu H, Wang Z, Dong B. PDEFomer: towards a foundation model for One-Dimensional Partial Differential equations. *arXiv:2402.12652v1* [Preprint]. 2024 [cited 2024 February 20]. Available from: <http://arxiv.org/abs/2402.12652v1>
35. Zerveas G, Jayaraman S, Patel D, Bhamidipaty A, Eickhoff C. A Transformer-based Framework for Multivariate Time Series Representation Learning. In: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021. p. 2114–24. <https://doi.org/10.1145/3447548.3467401>

36. Hajirahimi Z, Khashei M. Hybrid structures in time series modeling and forecasting: A review. *Eng Appl Artif Intell*. 2019;86:83–106. <https://doi.org/10.1016/j.engappai.2019.08.018>
37. Kim T, Kim J, Tae Y, Park C, Choi J, Choo J. Reversible instance normalization for accurate time-series forecasting against distribution shift. In: *International Conference on Learning Representations*. 2021. Available from: <https://openreview.net/pdf?id=cGDAkQo1C0p>
38. Shi X, Chen Z, Wang H, Yeung DY, Wong WK, Woo Wc. Convolutional LSTM network: A machine learning approach for precipitation nowcasting. *Adv Neural Inf Process Syst*. 2015;28.
39. Wang Y, Long M, Wang J, Gao Z, Yu PS. PredRNN: Recurrent neural networks for predictive learning using spatiotemporal LSTMs. *Adv Neural Inf Process Syst*. 2017;30.
40. Gao Z, Tan C, Wu L, Li SZ. Simvp: Simpler yet better video prediction. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2022. p. 3160–70. Available from: <https://doi.org/10.1109/cvpr52688.2022.00317>
41. Pathak J, Subramanian S, Harrington P, Raja S, Chattopadhyay A, Mardani M, et al. FourCastNet: A Global Data-driven High-resolution Weather Model using Adaptive Fourier Neural Operators. *arXiv:2202.11214v1 [Preprint]*. 2022 [cited 2022 February 22]. Available from: <https://arxiv.org/abs/2202.11214v1>
42. Lam R, Sanchez-Gonzalez A, Willson M, Wirsberger P, Fortunato M, Pritzel A, et al. GraphCast: Learning skillful medium-range global weather forecasting. *arXiv:2212.12794v1 [Preprint]*. 2022 [cited 2022 December 24]. Available from: <https://arxiv.org/abs/2212.12794v1>
43. Gasparin A, Lukovic S, Alippi C. Deep learning for time series forecasting: The electric load case. *CAAI Trans on Intel Tech*. 2021;7(1):1–25. <https://doi.org/10.1049/cit2.12060>
44. Zhou H, Zhang S, Peng J, Zhang S, Li J, Xiong H, et al. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. *AAAI*. 2021;35(12):11106–15. <https://doi.org/10.1609/aaai.v35i12.17325>